

E-Commerce Product Sorting System

A Case Study in Large-Scale Sorting Using Merge Sort

Student: ADITHYA R

Register Number: 192311369

Domain: Design and Analysis of Algorithms

Technique Used: Merge Sort — Divide and Conquer

1. Case Study Overview and Problem Analysis

An e-commerce platform maintains a catalog of millions of products, each with attributes such as price and customer rating. Users expect near-instant access to sorted views of this catalog — for example, ‘sort by price: low to high’ or ‘sort by customer rating’ — often combined with pagination (showing 20–50 products per page) and filters (category, brand, availability). The platform must therefore sort very large datasets reliably, repeatedly, and predictably, while preserving a sensible order among products that are tied on the chosen key (e.g., several products priced at exactly ₹499).

This case study analyzes Merge Sort as the sorting technique for this system: how its divide-and-conquer structure processes large datasets, why its stability property matters for multi-attribute product ordering, how it scales to catalogs too large for memory, and why it is a well-justified choice for this specific industry scenario.

1.1 Key Requirements Identified

- **Correctness at scale:** the algorithm must sort millions of records without a worst-case performance collapse, since product data can arrive in any order (freshly ingested, partially pre-sorted by a previous run, or adversarially patterned by bulk uploads).
- **Stability:** when two products tie on the primary sort key (e.g., identical price), their relative order should be predictable and consistent — typically preserving a meaningful secondary order (such as rating) rather than shuffling arbitrarily.
- **Scalability:** the solution must extend gracefully from an in-memory sort of a single category (thousands of items) to a catalog-wide sort (millions of items, potentially exceeding available RAM).
- **Predictable performance:** sorting is often on the critical path of a user-facing request or a nightly batch job with a fixed time window, so worst-case behavior must be as tightly bounded as average-case behavior.

2. Applying Divide and Conquer: How Merge Sort Processes the Catalog

Merge Sort solves the sorting problem using the classic divide-and-conquer paradigm, which breaks a large problem into smaller instances of the same problem, solves each independently, and combines their solutions:

1. **Divide:** split the current list of products into two halves of (nearly) equal size. This step does no comparison work — it is a pure partitioning of the index range.
2. **Conquer:** recursively apply Merge Sort to each half independently, until a sublist contains 0 or 1 products — a trivially sorted base case.
3. **Combine:** merge the two now-sorted halves into a single sorted list by repeatedly comparing the front elements of each half and appending the smaller (or, per the tie-breaking rule, the one that appeared first) to the output.

Applied to a product catalog, this means the platform never has to reason about the entire multi-million-item list at once. A batch of products is recursively halved down to individual items, and the sorted order is rebuilt

purely through pairwise merges — each merge step only ever looks at two already-sorted sequences, which keeps the per-step logic simple, testable, and easy to parallelize or move to disk-backed processing.

2.1 Pseudocode

```
function MERGE-SORT(products, left, right):
    if left >= right:
        return // base case: 0 or 1 element
    mid = (left + right) / 2
    MERGE-SORT(products, left, mid) // divide: sort left half
    MERGE-SORT(products, mid + 1, right) // divide: sort right half
    MERGE(products, left, mid, right) // combine: merge sorted halves

function MERGE(products, left, mid, right):
    leftHalf = copy of products[left .. mid]
    rightHalf = copy of products[mid+1 .. right]
    i = 0; j = 0; k = left
    while i < len(leftHalf) and j < len(rightHalf):
        if leftHalf[i].key <= rightHalf[j].key: // '<=' preserves stability
            products[k] = leftHalf[i]; i += 1
        else:
            products[k] = rightHalf[j]; j += 1
        k += 1
    copy any remaining elements of leftHalf / rightHalf into products
```

The single comparison $\text{leftHalf}[i].\text{key} \leq \text{rightHalf}[j].\text{key}$ (using $\leq$ rather than $<$) is the entire mechanism behind Merge Sort's stability: whenever two products tie on the sort key, the element from the left half — which, by construction, appeared earlier in the original sequence — is always chosen first, so equal-key elements never swap their relative order.

3. Worked Example — Divide and Conquer Trace

Consider sorting 7 product prices: [38, 27, 43, 3, 9, 82, 10]. The recursion divides this array down to single elements, then merges back up:

Stage	Resulting Sequences
Divide 1	[38, 27, 43] [3, 9, 82, 10]
Divide 2	[38] [27, 43] [3, 9] [82, 10]
Divide 3	[38] [27] [43] [3] [9] [82] [10]
Merge 1	[38] [27,43] [3,9] [10,82]
Merge 2	[27, 38, 43] [3, 9, 10, 82]
Merge 3 (final)	[3, 9, 10, 27, 38, 43, 82]

Every merge step only ever combines two already-sorted sequences, and the recursion depth is $\log_2(7) \approx 3$ levels — for a catalog of a million products, the same structure needs only $\log_2(1,000,000) \approx 20$ levels of division, after which merging back up performs a total of $O(n \log n)$ comparisons regardless of the input order.

4. Stability — Why It Matters for Product Sorting

A sorting algorithm is stable if it preserves the relative order of elements that compare as equal on the sort key. Merge Sort is stable by construction (Section 2.1). This property is not a minor technicality for an e-commerce platform — it directly enables correct multi-attribute sorting and consistent pagination.

4.1 Multi-Key Sorting via Stability

Suppose the platform wants products ordered primarily by price (ascending) and, among products with the same price, by customer rating (descending). A stable sort makes this achievable with two simple passes: first sort by the secondary key (rating, descending), then stable-sort the result by the primary key (price, ascending). Because the second pass never disturbs the relative order of equal-price items, the rating-descending order established in the first pass survives intact within each price group.

Product	Price (₹)	Rating	Original Position
P1	500	4.5	1
P2	300	4.0	2
P3	500	3.8	3
P4	300	4.5	4
P5	700	4.2	5
P6	500	4.0	6

4.2 Pass 1: Stable Sort by Rating (Descending)

Result: P1(4.5), P4(4.5), P5(4.2), P2(4.0), P6(4.0), P3(3.8) — note P1 stays before P4 (both 4.5) and P2 stays before P6 (both 4.0), since Merge Sort preserves their original relative order on this tie.

4.3 Pass 2: Stable Sort of That Result by Price (Ascending)

Result: P4(₹300, 4.5), P2(₹300, 4.0), P1(₹500, 4.5), P6(₹500, 4.0), P3(₹500, 3.8), P5(₹700, 4.2).

Within the ₹300 group, P4 (4.5) correctly precedes P2 (4.0); within the ₹500 group, P1 (4.5), P6 (4.0), and P3 (3.8) appear in correct descending-rating order — the secondary ordering from Pass 1 was fully preserved through Pass 2 purely because the sort used is stable.

If an unstable algorithm (e.g., a naive Quick Sort or Heap Sort) were substituted for Pass 2, ties on price could be reordered arbitrarily by the partitioning/heap logic, silently destroying the rating order established in Pass 1 — e.g., P6 and P3 could swap positions with no error raised, showing a lower-rated item above a higher-rated

one within the same price bracket. This is a subtle correctness bug that would only surface as an inconsistent user experience, not a crash.

4.4 Pagination Consistency

Stability also protects pagination. If a user requests page 2 of ‘sort by price’ results a few seconds after page 1, and the underlying sort is unstable, ties on price could be reordered between the two requests (especially if the catalog changed slightly, or if the sort is recomputed per request), causing a product to appear on both pages or on neither. A stable sort keyed consistently on (price, product ID) as a tiebreaker guarantees the same product order across repeated requests, which is essential for a coherent browsing experience.

5. Scalability — Handling Millions of Products

5.1 In-Memory Merge Sort (Single Machine, Fits in RAM)

For catalogs or category-level subsets that fit comfortably in memory (up to tens of millions of lightweight records), standard recursive Merge Sort as described in Section 2 is sufficient, completing in $O(n \log n)$ time with a predictable, input-independent performance profile.

5.2 External Merge Sort (Catalog Exceeds Available RAM)

When the full catalog is too large to fit in memory, the platform can use External Merge Sort, a direct extension of the same divide-and-conquer idea to disk-backed storage:

4. Run generation: read the catalog in memory-sized chunks (e.g., 500 MB at a time), sort each chunk in memory using ordinary Merge Sort, and write each sorted chunk back to disk as a ‘run’.
5. K-way merge: open all sorted runs simultaneously and merge them using a min-heap keyed on the current front element of each run, repeatedly extracting the smallest and writing it to the final output — a direct generalization of the two-way merge in Section 2.1 to k sorted inputs.
6. Repeat merging in passes if the number of runs exceeds the number of files that can be open at once, until a single fully sorted output file remains.

This is exactly the technique used internally by database systems and search-index builders (e.g., segment merging in Lucene/Elasticsearch) for sorting or building indexes over datasets far larger than RAM — and it is a natural fit here specifically because Merge Sort's combine step already only requires sequential, one-directional access to its inputs, which maps cleanly onto sequential disk/stream reads.

5.3 Parallel and Distributed Merge Sort

- Multi-threaded divide: because the two recursive halves in Section 2.1 are fully independent, they can be sorted on separate CPU cores in parallel with no shared-state coordination needed until the final merge.
- Distributed (MapReduce-style) sort: partition the catalog across multiple machines, sort each partition locally (map phase), then perform a distributed k -way merge of the partitions' sorted outputs (reduce phase) — the same conceptual structure as Section 5.2, spread across a cluster.

- Incremental updates: when new products are added or prices change, avoid re-sorting the entire catalog — sort only the small batch of new/changed items, then merge that already-sorted small batch into the existing large sorted list with a single linear-time merge pass (Section 2.1's MERGE step applied once), which is far cheaper than a full $O(n \log n)$ re-sort.

5.4 Practical Serving Layer

- Precompute and cache sorted indexes for the most common sort orders (price ascending/descending, rating descending, popularity) so that most user requests are served from a ready-made sorted structure rather than sorting on every request.
- Refresh these cached sorted indexes using the incremental merge technique above whenever the underlying catalog changes, rather than recomputing from scratch.

6. Time and Space Complexity Analysis

6.1 Time Complexity

Let $T(n)$ be the time to sort n products. The recurrence is $T(n) = 2T(n/2) + O(n)$, where the $O(n)$ term is the cost of merging two sorted halves of total size n . By the Master Theorem (case 2: $a=2$, $b=2$, $f(n)=O(n)$ matches $n^{(\log_b a)} = n^1$), this recurrence solves to $T(n) = O(n \log n)$ — identically in the best, average, and worst case, because the divide step always splits evenly regardless of the input's initial order, and the merge step always performs a linear scan regardless of how sorted or unsorted the halves were relative to each other.

Algorithm	Best Case	Average Case	Worst Case
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

This worst-case guarantee is significant for a production sorting service: Quick Sort degrades to $O(n^2)$ on already-sorted or adversarially patterned input (a realistic risk, since bulk product uploads or previously-sorted exports could arrive in near-sorted order), whereas Merge Sort's performance is entirely input-order-independent — making it far easier to reason about and guarantee SLA-bound sorting latency for a catalog service.

6.2 Space Complexity

Merge Sort requires $O(n)$ auxiliary space for the temporary arrays used during merging (as seen in the MERGE function in Section 2.1, which allocates leftHalf and rightHalf copies). This contrasts with Quick Sort and Heap Sort, which sort in place using only $O(\log n)$ or $O(1)$ auxiliary space respectively.

Algorithm	Space Complexity	Stable?
Merge Sort	$O(n)$	Yes

Quick Sort	$O(\log n)$	No
Heap Sort	$O(1)$	No

The $O(n)$ space cost is Merge Sort's principal drawback and the direct motivation for the External Merge Sort design in Section 5.2 — rather than treating $O(n)$ auxiliary memory as prohibitive, the system bounds it to a fixed, memory-sized chunk per run and shifts the aggregation work to a streaming k-way merge, so the memory footprint stays constant regardless of total catalog size.

7. Feasible Solution Design

Combining the analysis above, a practical sorting subsystem for the e-commerce platform can be designed as follows:

7. Attribute normalization: ensure every product record has well-defined, comparable price and rating fields (consistent currency/rounding for price; a fixed decimal precision for rating), plus a unique, immutable product ID reserved as a final tiebreaker key to guarantee fully deterministic ordering.
8. Batch (offline) sort: run External Merge Sort (Section 5.2) over the full catalog on a nightly or event-triggered schedule, producing sorted indexes for each supported sort order (price ascending, price descending, rating descending), using the stable multi-pass technique from Section 4.1 wherever a secondary tiebreak key is required.
9. Incremental (online) update: as individual products are added, removed, or re-priced during the day, apply the incremental merge technique (Section 5.3) to fold small batches of changes into the existing sorted indexes without a full re-sort.
10. Parallel execution: distribute the run-generation and k-way merge phases of the batch sort across available cores or cluster nodes (Section 5.3) to keep the nightly batch job within its time window as the catalog grows.
11. Serving layer: cache the resulting sorted indexes and serve paginated, filtered views directly from them, re-applying any category/brand filters as a pass over the pre-sorted index rather than sorting per request.

This design keeps the per-request cost of returning a sorted, paginated product list to a simple lookup against a pre-built sorted structure, while confining the $O(n \log n)$ sorting cost to controlled batch/incremental jobs where Merge Sort's predictable performance and stability guarantees are exploited directly.

8. Why Merge Sort Is Suitable for This Scenario

- Guaranteed $O(n \log n)$ performance regardless of input order removes the risk of a worst-case slowdown that Quick Sort exposes on sorted/near-sorted bulk uploads — critical for a service with latency/SLA expectations.
- Built-in stability directly enables correct, predictable multi-attribute sorting (price then rating) using simple, well-understood multi-pass sorting, and keeps paginated results consistent across requests —

both are concrete, user-facing correctness requirements for an e-commerce catalog, not just theoretical nice-to-haves.

- The clean divide-and-conquer structure generalizes directly and naturally to external (disk-based), parallel, and incremental sorting — exactly the three scalability techniques (Sections 5.2–5.3) needed to grow from a single in-memory sort to a catalog of millions of products.
- The $O(n)$ space overhead, its main theoretical weakness, is manageable in this context because the batch/external-sort design already processes the catalog in fixed-size chunks rather than loading it wholesale, so Merge Sort's memory requirement never becomes the bottleneck in practice.

Taken together, Merge Sort's predictability, stability, and natural fit with external/parallel/incremental extensions make it a well-justified default choice for the batch and incremental sorting layers of an e-commerce product catalog, with the per-request serving layer relying on the pre-sorted output rather than re-sorting on demand.

9. Conclusion

Sorting millions of e-commerce products by price and rating is a large-scale, repeatedly-executed operation with strict correctness and performance expectations. Merge Sort's divide-and-conquer structure delivers a guaranteed $O(n \log n)$ time bound independent of input order, its stability property enables correct multi-key sorting and consistent pagination, and its clean recursive structure extends naturally to external, parallel, and incremental sorting techniques needed at real catalog scale. While its $O(n)$ auxiliary space requirement is a genuine trade-off compared to in-place alternatives like Quick Sort or Heap Sort, this cost is well managed through chunked external sorting, making Merge Sort a sound and well-justified foundation for the platform's product sorting system.